
REAL NEW YORK

AI Chatbot & Administrative Dashboard Console

Author: Product Engineering Team

Status: Release Documentation

API Target: Google Gemini API (gemini-3.5-flash)

Deployment: Client-Side SPA (Static Web Hosting)

Date: July 2026

CONFIDENTIAL | INTERNAL USE ONLY

1. Executive Summary

This document describes the design, implementation, and capabilities of the **REAL New York AI Chatbot Console & Dashboard Application**. The application provides a premium, interactive simulation of virtual real estate advisory services for rentals, sales, and commercial leasing, enabling agents to test client interactions and personalize chatbot personas in real time.

Built on a client-side stack to ensure rapid responsiveness, the platform leverages the **Google Gemini API** (model: gemini-3.5-flash) to deliver context-aware replies grounded in standard NYC rental rules and an in-memory listings catalog database.

2. System Architecture

The application is engineered as a zero-dependency, single-page application (SPA) to maximize efficiency, loading speeds, and styling flexibility.

File	Role	Key Modules / Contents
index.html	Structure & Layout	HTML5 structure, admin tab layouts, chatbot dialog panels, and inline modals.
styles.css	Aesthetics & Themes	Design tokens, CSS variables, glassmorphism card modules, light/dark modes, and custom
app.js	Logic & API Bridge	State management, mock listings and leads databases, REST calls to Gemini model API,

3. Chatbot Personas

To address different divisions of a full-service brokerage, the console supports three distinct agent personas. Selecting a persona updates the interface's color accent, welcome greeting, and suggestion chips instantly:

Persona	Specialty Division	UI Accent Color	Greeting Theme
Emma	Residential Rentals	#ff5e36 (REAL Orange)	Rentals focus: 40x rule, neighborhood searches, LIC, William
Alex	Commercial Advisor	#00b0ff (Electric Blue)	Business spaces: storefronts, office suites, lease terms.
Sophia	VIP Concierge	#d500f9 (Purple Spark)	Luxury developments: purchase condos, penthouses, priva

4. Gemini API Integration Spec

The live agent replies are served by querying the gemini-3.5-flash endpoint via HTTP POST. The model is supplied with a custom system instruction prompt and listing context to simulate real-estate advice.

Gemini Multi-Turn Role Alternation Rule

The Gemini REST API requires a strict alternating sequence in the conversation history (i.e. user, model, user, model...) and mandates that the history list **must begin with a user turn**. Because the bot starts with a greeting (a model turn), this was causing validation failures (status 400).

```
const recentHistory = conversationHistory.slice(-10);
const contents = [];
let expectedRole = "user";
for (const msg of recentHistory) {
  const role = msg.role === "user" ? "user" : "model";
  if (role === expectedRole) {
    contents.push({
      role: role,
      parts: [{ text: msg.text }]
    });
    expectedRole = expectedRole === "user" ? "model" : "user";
  }
}
```

Implementation Note: The code snippet above scans the history array and selectively builds a compliant alternating payload, stripping out initial model greetings or double-sent messages dynamically before dispatching.

5. Console Panels & Features

A. Bot Customizer Tab

This panel serves as the controls module. Agents can adjust greetings and suggestions on-the-fly:

- **Custom Greeting:** A textarea that updates the live chat preview immediately.
- **Suggestions Chips Editor:** Input fields that allow setting dynamic quick-select chips under the chat box.
- **API Key Integration:** An input block with a status display indicator ('Active' vs 'Offline Rules Fallback'). Input keys are written to the browser's localStorage, bypassing the remote repository to ensure zero security leaks.

B. Leads Captured Database

Logs client details gathered by the chatbot's dynamic forms:

- **Fields Recorded:** Timestamp, client name, contact details (email/phone), requirements/dates, and status badge.
- **Lead Types:** Displays Tour Scheduled (in green) or Info Only (in blue).
- **Analytics Cards:** Aggregates total captured leads count, tour requests, and tracks the top requested neighborhood.

C. Mock Listings Database

Provides the chatbot with its catalog of properties. Agents can query, add, or remove options in real time. When the user asks for listings matching specific locations or budgets, the local matcher filters these entries to return cards.

6. Interactive Chatbot UI Widgets

The model response parsing script checks for special markers to insert dynamic visual components directly in the stream:

Widget Tag	Render Output	Action & Lead Capture Flow
[CAROUSEL: id1, id2]	Horizontal Card Carousel	Generates visual property cards containing rents, spec icons (beds/ba
[SCHEDULER]	Inline Tour Calendar Widget	Generates a viewing request form. Upon submission, it registers the cl
[LEAD_FORM]	Client Profile Registration Card	Generates a client contact capture card. Upon submission, it creates a

7. Local Rules Engine (Offline Fallback)

If no API Key is entered or if the API request fails due to network conditions, the console seamlessly falls back to a **Local Keyword Rules Engine**. This fallback parses basic expressions (e.g. 'schedule', '40x rule', 'guarantors', 'documents', or specific numbers/neighborhoods) to return pre-formatted FAQs or query the mock listing array directly, providing an uninterrupted user experience.